I. Objetivos

	**UNIVERSIDAD DON BOSCO** **FACULTAD DE INGENIERIA** **ESCUELA DE COMPUTACION**
CICLO 02	**GUIA DE LABORATORIO Nº 4**
	Nombre de la practica: Introducción al Lenguaje SQL **Lugar de ejecución:** Laboratorio de Informática **Tiempo estimado:** 2 horas **Materia:** Análisis y diseño de sistemas y base de datos

1. Implementar las herramientas de manipulación de una base de datos en SQL Management Studio
2. Identificar los diferentes objetos dentro de una base de datos
3. Definir tipos de datos con Lenguaje SQL
4. Crear, alterar y eliminar tablas y base de datos con Lenguaje SQL

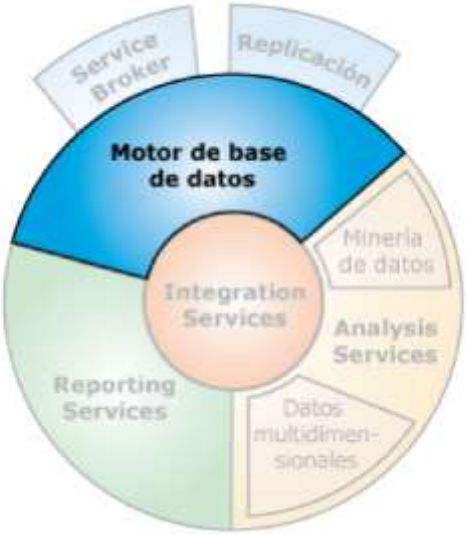
II. Introducción Teórica

Motor de base de datos de SQL Server

El Motor de base de datos es el servicio principal para almacenar, procesar y proteger datos. El Motor de base de datos proporciona acceso controlado y procesamiento de transacciones rápido para cumplir con los requisitos de las aplicaciones consumidoras de datos más exigentes de una empresa.

El Motor de base de datos se utiliza para crear bases de datos relacionales para el procesamiento de transacciones en línea o datos de procesamiento analítico en línea. Esto incluye la creación de tablas para almacenar datos y objetos de base de datos (p.ej., índices, vistas y procedimientos almacenados) para ver, administrar y proteger datos.

Se puede usar SQL Server Management Studio para administrar los objetos de bases de datos y SQL Server Profiler para capturar eventos de servidor.



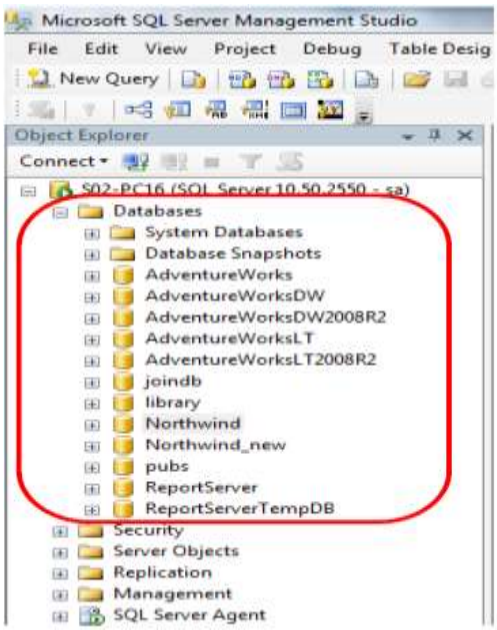
Fundamentos de bases de datos

Una base de datos de SQL Server consta de una colección de tablas en las que se almacena un conjunto específico de datos estructurados. Una tabla contiene una colección de filas, también denominadas tuplas o registros, y columnas, también denominadas atributos. Cada columna de la tabla se ha diseñado para almacenar un determinado tipo de información; por ejemplo, fechas, nombres, importes en moneda o números.

Las tablas contienen diversos tipos de controles, como restricciones, desencadenadores, valores predeterminados y tipos de datos de usuario personalizados, que garantizan la validez de los datos. Se pueden agregar restricciones de integridad referencial declarativa a las tablas con el fin de asegurar la coherencia de los datos interrelacionados que se encuentran en tablas distintas.

Las tablas pueden presentar índices, similares a los de los libros, que permiten localizar las filas rápidamente.

Una instancia de SQL Server admite muchas bases de datos. Cada base de datos puede almacenar datos interrelacionados o sin relacionar procedentes de otras bases de datos. Por ejemplo, una instancia de SQL Server puede tener una base de datos que almacene datos del personal y otra que almacene datos relacionados con los productos. Por otra parte, puede utilizarse una base de datos para almacenar datos acerca de pedidos actuales de los clientes y otra base de datos relacionada puede almacenar pedidos anteriores de los clientes que se utilicen para la elaboración de los informes anuales.



Tipos de datos

En SQL Server, cada columna, variable local, expresión y parámetro tiene un tipo de datos relacionado. Un tipo de datos es un atributo que especifica el tipo de datos que el objeto puede contener: datos de enteros, datos de caracteres, datos de moneda, datos de fecha y hora, cadenas binarias, etc.

Categorías de tipos de datos

Los tipos de datos de SQL Server se organizan en las siguientes categorías:

Numéricos exactos	Cadena de caracteres Unicode
Numéricos aproximados	Cadenas binarias
Fecha y hora	Otros tipos de datos
Cadena de caracteres	

En SQL Server, según las características de almacenamiento, algunos tipos de datos están designados como pertenecientes a los siguientes grupos:

* Tipos de datos de valores grandes:
  varchar(max), nvarchar(max) y **varbinary(max)**
* Tipos de datos de objetosgrandes:
  text, ntext, image, varchar(max), nvarchar(max), varbinary(max) y **xml**

Numéricos exactos

bigint	numeric
bit	smallint
decimal	smallmoney
int	tinyint
money	

Numéricos aproximados

float	real

Fecha y hora

date	datetimeoffset
datetime2	smalldatetime
datetime	time

Cadena de caracteres

char	varchar
text	

Cadena de caracteres Unicode

nchar	nvarchar
ntext	

Cadenas binarias

binary	varbinary
image	

Otros tipos de datos

cursor	timestamp
hierarchyid	uniqueidentifier
sql_variant	xml
tabla	

Lenguaje SQL

SQL (Structured Query Language), Lenguaje Estructurado de Consulta es el lenguaje utilizado para definir, controlar y acceder a los datos almacenados en una base de datos relacional.
Como ejemplos de sistemas gestores de bases de datos que utilizan SQL podemos citar DB2, SQL Server, Oracle, MySql, Sybase, PostgreSQL o Access.

El SQL es un lenguaje universal que se emplea en cualquier sistema gestor de bases de datos relacional. Tiene un estándar definido, a partir del cual cada sistema gestor ha desarrollado su versión propia.

En SQL Server la versión de SQL que se utiliza se llama TRANSACT-SQL.

Se puede ejecutar directamente en modo interactivo, pero también se suele emplear embebido en programas escritos en lenguajes de programación convencionales. En estos programas se mezclan las instrucciones del propio lenguaje (denominado anfitrión) con llamadas a procedimientos de acceso a la base de datos que utilizan el SQL como lenguaje de acceso. Como por ejemplo en Visual Basic, Java, C#, PHP .NET, etc.

Las instrucciones SQL se clasifican según su propósito en tres grupos:

* El DDL (Data Description Language) Lenguaje de Descripción de Datos.
* El DCL (Data Control Language) Lenguaje de Control de Datos.
* El DML (Data Manipulation Language) Lenguaje de Manipulación de Datos.

El DDL, es la parte del SQL dedicada a la definición de la base de datos, consta de sentencias para **definir la estructura de la base de datos**, permiten crear la base de datos, crear, modificar o eliminar la estructura de las tablas, crear índices, definir reglas de validación de datos, relaciones entre las tablas, etc. Permite definir gran parte

del nivel interno de la base de datos. Por este motivo estas sentencias serán utilizadas normalmente por el administrador de la base de datos.

El DCL (Data Control Language) se compone de instrucciones que permiten:
Ejercer un control sobre los datos tal como la asignación de privilegios de acceso a los datos (GRANT/REVOKE).

El DML se compone de las **instrucciones para el manejo de los datos**, para insertar nuevos datos, modificar datos existentes, para eliminar datos y la más utilizada, para recuperar datos de la base de datos. Veremos que una sola instrucción de recuperación de datos es tan potente que permite recuperar datos de varias tablas a la vez, realizar cálculos sobre estos datos y obtener resúmenes.

Es el lenguaje que utilizan los programadores y los usuarios de la base de datos.

La gestión de transacciones (COMMIT/ROLLBACK).
Una transacción se puede definir como un conjunto de acciones que se tienen que realizar todas o ninguna para preservar la integridad de la base de datos.

COMMIT
Si una transacción es válida, se hace.
La instrucción **COMMIT** garantiza que todas las modificaciones de la transacción se conviertan en una parte permanente de la base de datos. La instrucción COMMIT también libera recursos que utiliza la transacción como, por ejemplo, los bloqueos.
Hay que utilizar **COMMIT TRANSACTION** solo en el punto donde todos los datos a los que hace referencia la transacción sean lógicamente correctos.

ROLLBACK
Si se produce un error en una transacción o el usuario decide cancelar la transacción, hay que echar para tras la transacción. Esto se hace con la instrucción **ROLLBACK** que devuelve los datos al estado en que estaban al inicio de la transacción. La instrucción ROLLBACK también libera los recursos que mantiene la transacción.

Referencia de Transact-SQL (motor de base de datos)
Transact-SQL es fundamental para trabajar con SQL Server. Todas las aplicaciones que se comunican con SQL Server lo hacen enviando instrucciones Transact-SQL al servidor, independientemente de la interfaz de usuario de la aplicación.
A continuación, se proporciona una lista de las aplicaciones que pueden generar Transact-SQL:

- Aplicaciones generales de productividad en oficinas.
- Aplicaciones que utilizan una interfaz gráfica de usuario (GUI) para permitir al usuario seleccionar las tablas y columnas cuyos datos desea ver.
- Aplicaciones que utilizan instrucciones del lenguaje general para determinar los datos que el usuario desea ver.
- Aplicaciones de la línea de negocios que almacenan sus datos en bases de datos SQL Server. Estas aplicaciones pueden incluir aplicaciones de otros proveedores o escritas internamente.
- Scripts Transact-SQL que se ejecutan con herramientas tales como **sqlcmd**.
- Aplicaciones creadas con sistemas de desarrollo tales como Microsoft Visual C++, Microsoft Visual Basic o Microsoft Visual J++, y que usan API de base de datos tales como ADO, OLE DB y ODBC.
- Páginas web que extraen datos de bases de datos SQL Server.
- Sistemas de bases de datos distribuidos desde los que se replican datos SQL Server en varias bases de datos o se ejecutan consultas distribuidas.
- Almacenamientos de datos en los que los datos se extraen de los sistemas de procesamiento de transacciones en línea (OLTP) y se resumen para el análisis dirigido a la toma de decisiones.

Instrucción CREATE
Es usada para la creación de objetos dentro de SQL, siendo posible crear bases de datos, tablas, etc.

CREATE <tipo de objeto> <nombre del objeto>
A esta parte le seguirán los detalles, que variarán según la naturaleza del objeto que estemos creando. A continuación, se presenta un listado de sintaxis más completa de CREATE:

CREATE DATABASE
Crea una nueva base de datos y los archivos que se utilizan para almacenar la base de datos
Sintaxis:

```
CREATE DATABASE <nombre de base de datos>
ON PRIMARY
(NAME = <nombre lógico del archivo>,
 FILENAME = <nombre del archivo>,
 SIZE = <tamaño en Kilobytes, megabytes, gigabytes, o terabytes>,
 MAXSIZE = <tamaño en Kilobytes, megabytes, gigabytes, o terabytes>,
 FILEGROWTH = <tamaño en Kilobytes, megabytes, gigabytes, o terabytes| porcentaje>)
LOG ON
(NAME = <nombre lógico del archivo>,
 FILENAME = <nombre del archivo>,
 SIZE = <tamaño en Kilobytes, megabytes, gigabytes, o terabytes>,
 MAXSIZE = <tamaño en Kilobytes, megabytes, gigabytes, o terabytes>,
 FILEGROWTH = <tamaño en Kilobytes, megabytes, gigabytes, o terabytes| porcentaje>);
 COLLATE <nombre de intercalación>
```

Algunas de las opciones anteriores son mutuamente excluyentes (por ejemplo, si está creando para anexar, la mayoría de las opciones que no sean ubicaciones de archivo no serán válidas). En esta sintaxis hay mucho que explicar, por lo que vamos a desglosarla en sus elementos.

ON:
Se utiliza en dos sitios para definir la ubicación del archivo donde se almacenan los datos (Archivo .MDF) y para definir la misma información para el lugar donde se guarda el registro (Log de transacciones, archivo .LDF). Advertirá aquí la inclusión de la palabra clave PRIMARY, que indica que lo que sigue es un grupo de archivos primarios (o principales) en el que se guardan físicamente los datos. También podemos guardar datos en los denominados grupos de archivos secundarios.

NAME:
Éste es el nombre del archivo que estamos definiendo, pero sólo es un nombre lógico, es decir, el nombre que va a utilizar SQL Server internamente para hacer referencia a dicho archivo.

FILENAME:
Éste es el nombre físico del disco del archivo del sistema operativo real en el que se van a guardar los datos y el registro (Log de transacciones). El valor predeterminado dependerá de si estamos tratando con la propia base de datos o con el Log de transacciones. De forma predeterminada, el archivo se ubicará en el siguiente subdirectorio \ Data dentro del directorio C:\ Archivos de programa \ Microsoft SQLServer \ MSSQL.1 \ MSSQL (o el directorio que ha ya establecido como principal para SQL Server en la instalación). Si estuviésemos utilizando el archivo de base de datos físico, se denominaría igual que nuestra base de datos con una extensión .mdf y si estuviésemos utilizando el registro, tendría el mismo nombre que el archivo de base de datos, pero con un sufijo _log y una extensión .ldf.

SIZE:
Es el tamaño de la base de datos. De forma predeterminada, el tamaño se proporciona en megabytes, pero también podemos hacer que se proporcione en kilobytes utilizando KB en lugar de MB tras el valor numérico del tamaño; también podemos usar un tamaño mucho mayor con GB (gigabytes) o incluso TB (terabytes). Tenga en cuenta que este valor debe ser, al menos, tan alto como el de la base de datos model y debe ser un número entero (sin decimales); en caso contrario recibiremos un error. Si no suministramos un valor para SIZE, la base de datos tendrá inicialmente el mismo tamaño que el de la base de datos model.

MAXSIZE:
Este parámetro es una pequeña variante del parámetro SIZE. SQL Server tiene un mecanismo que permite a nuestra base de datos asignar automáticamente un espacio en disco adicional (para crecer) cuando sea necesario. MAXSIZE es el tamaño máximo al que puede crecer la base de datos. Una vez más, de forma predeterminada se proporciona en megabytes, como SIZE, podemos utilizar KB, GB o TB para emplear cantidades de incremento diferentes. La pequeña variante es que no existe un valor predeterminado firme.
Si nuestra base de datos (el archivo .mdf) llega al valor establecido en el parámetro MAXSIZE, nuestros usuarios empezarán a recibir errores indicando que sus inserciones no se pueden ejecutar. Si nuestro registro (Log de transacciones .ldf) llega a su tamaño máximo, no podremos ejecutar ninguna actividad de inicio de sesión en la base de datos.

FILEGROWTH:
Mientras SIZE establece el tamaño inicial de la base de datos y MAXSIZE determina exactamente el tamaño máximo que puede llegar a tener el archivo de la base de datos, FILEGROWTH determina básicamente el incremento del crecimiento con que se puede llegar a dicho máximo. Para ello, proporcionamos un valor indicando por cuantos bytes (en KB, MB, GB, o TB) a la vez deseamos aumentar el archivo. Por ejemplo, si establecemos un archivo de base de datos para que cuando llegue a 1GB incremente en un valor FILEGROWTH de un 20%, la primera vez que se expanda, aumentará hasta a 1.2GB, la segunda vez hasta 1.44GB y así sucesivamente.

LOG ON:
Permite establecer que deseamos que nuestro registro (Log de transacciones) se dirija a un conjunto específico de archivos y dónde se deben ubicar exactamente dichos archivos. Si no proporcionamos esta opción, SQL Server creará el registro (Log de transacciones) en un solo archivo y lo predeterminará para que tenga un tamaño igual al 25% del tamaño del archivo de datos.

COLLATE:
Esta opción tiene que ver con el problema de la ordenación, las mayúsculas y minúsculas y los acentos. Al instalar su SQL Server, habrá decidido sobre cuál es la intercalación predeterminada, pero puede sobrescribir este parámetro a nivel de base de datos y a nivel de columna.

Recomendaciones
Es muy recomendable que guarde sus archivos de registro (.ldf) en una unidad de disco diferente a la de sus archivos de datos principales (.mdf). Al hacerlo, evitará que los archivos de datos principal y de registro compitan por la E/S del disco además de proporcionar una seguridad adicional si falla una unidad.

CREATE TABLE
Crea una nueva tabla en SQL Server
Sintaxis:

```
CREATE TABLE nombre_tabla
(
nombre_campo_1 tipo_1
nombre_campo_2 tipo_2
nombre_campo_n tipo_n
Key(campo_x,...)
)
```

RESTRICCIONES EN UNA TABLA

Limitar el tipo de dato que puede ingresarse en una tabla. Dichas restricciones pueden especificarse cuando la tabla se crea por primera vez a través de la instrucción **CREATE TABLE**, o luego de crear la tabla a través de la instrucción **ALTER TABLE**.

Los tipos comunes de restricciones incluyen las siguientes:

- NOT NULL
- UNIQUE
- CHECK
- Clave primaria
- Clave externa

NOT NULL

En forma predeterminada, una columna puede ser NULL. Si no desea permitir un valor NULL en una columna, deberá colocar una restricción en esta columna especificando que NULL no es ahora un valor permitido.

Por ejemplo, en la siguiente instrucción,

```
CREATE TABLE Cliente
(Codigo integer NOT NULL,
Apellido varchar (30) NOT NULL,
Nombre varchar(30));
```

Con la instrucción NOT NULL en las columnas “Codigo” y “Apellido”, estas no aceptan valores nulos (vacíos), mientras que el campo “Nombre” si puede contener valores nulos.

UNIQUE

La restricción UNIQUE asegura que todos los valores en una columna sean distintos.

Por ejemplo, en la siguiente instrucción,

```
CREATE TABLE Cliente
(Codigo integer UNIQUE,
Apellido varchar (30),
Nombre varchar(30));
```

La columna “Codigo” no puede incluir valores duplicados, dicha restricción no se aplica para columnas “Apellido” y “Nombre”.

Nota: esta restricción no sustituye la clave primaria

CHECK

La restricción CHECK asegura que todos los valores en una columna cumplan ciertas condiciones.

Por ejemplo, en la siguiente instrucción:

```
CREATE TABLE Customer
(Codigo integer CHECK (Codigo > 0),
```

```
Apellido varchar (30),
Nombre varchar(30));
```

La columna “Codigo” sólo acepta datos enteros mayores a 0.

ALTER TABLE

Modifica una definición de tabla al alterar, agregar o quitar columnas y restricciones, reasignar particiones, o deshabilitar o habilitar restricciones y desencadenadores.
Algunas instrucciones pueden ser:

Agregar un nuevo campo

Para agregar un nuevo campo a la tabla digitamos la siguiente sentencia:

```
ALTER TABLE CONTACTOS
ADD estado VARCHAR (8)
```

Esta sentencia nos permite agregar el campo estado con un tipo de dato VARCHAR de 8 caracteres a la tabla contactos.

Eliminar un campo

Para eliminar un campo de la tabla digitamos la siguiente sentencia

```
ALTER TABLE CONTACTOS
DROP COLUMN estado
```

AGREGAR UNA RESTRICCIÓN

Con la instrucción ALTER TABLE se pueden agregar las diferentes tipos de restricciones mencionadas anteriormente, por ejemplo:

Restricción Check

```
ALTER TABLE CONTACTOS
ADD CONSTRAINT exd_check CHECK (codigo> 1);
```

Restricción Default

```
ALTER TABLE VENTA
ADD CONSTRAINT default_fecha
DEFAULT getdate() FOR fechaventa ;
```

Llave Primaria (PRIMARY KEY)

```
ALTER TABLE CONTACTOS
ADD CONSTRAINT primary_codigo PRIMARY KEY CLUSTERED (Codigo);
```

Llave Foránea (FOREIGN KEY)

```
ALTER TABLE VENTA
ADD CONSTRAINT fK_venta FOREIGN KEY (IdVenta)
REFERENCES CONTACTOS (Codigo)
```

III. Requerimientos

- Maquina con SQL Server 2012, 2014, 2016 y 2019
- Guía Número 4 Introducción a Lenguaje SQL.

IV. Procedimiento

Parte 1: Iniciando sesión desde SQL Server Management Studio

1. Hacer clic en el botón **Inicio o búsqueda**
2. Hacer clic en la opción Todos los programas y hacer clic en **Microsoft SQL Server 2012 o 2016**

Para conectarse con el servidor de base de datos elija los siguientes parámetros de autenticación:

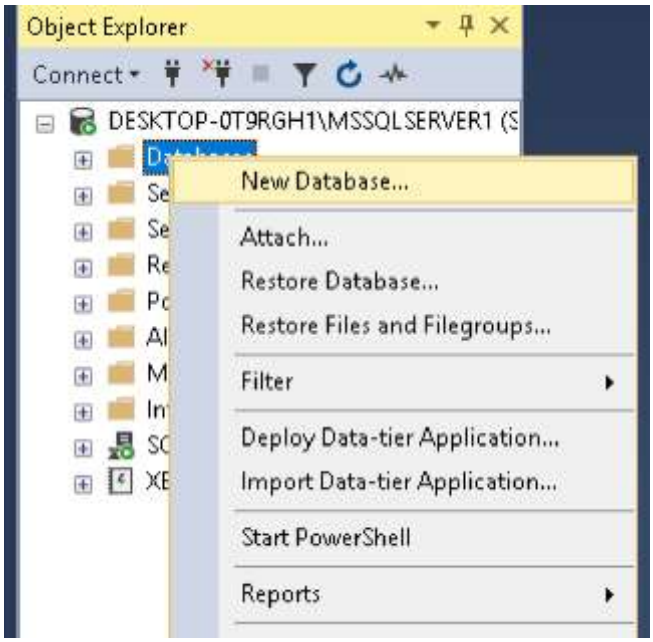
- **Server type (Tipo de servidor):** Database Engine
- **Server name (Nombre del servidor):** Colocar el nombre del servidor local, por ejemplo PC**NumMaquina- SALA2 o** SalaX-**NumMaquina**
  Nota: NumMaquina es el número de la maquina local
- **Authentication (Autenticación)**: SQL Server Authentication
- **Usuario (Login):** sa
- **Contraseña (Password):** 123456

NOTA: En caso de surgir problemas con SQL Server Authentication puede recurrir a Windows Autentication

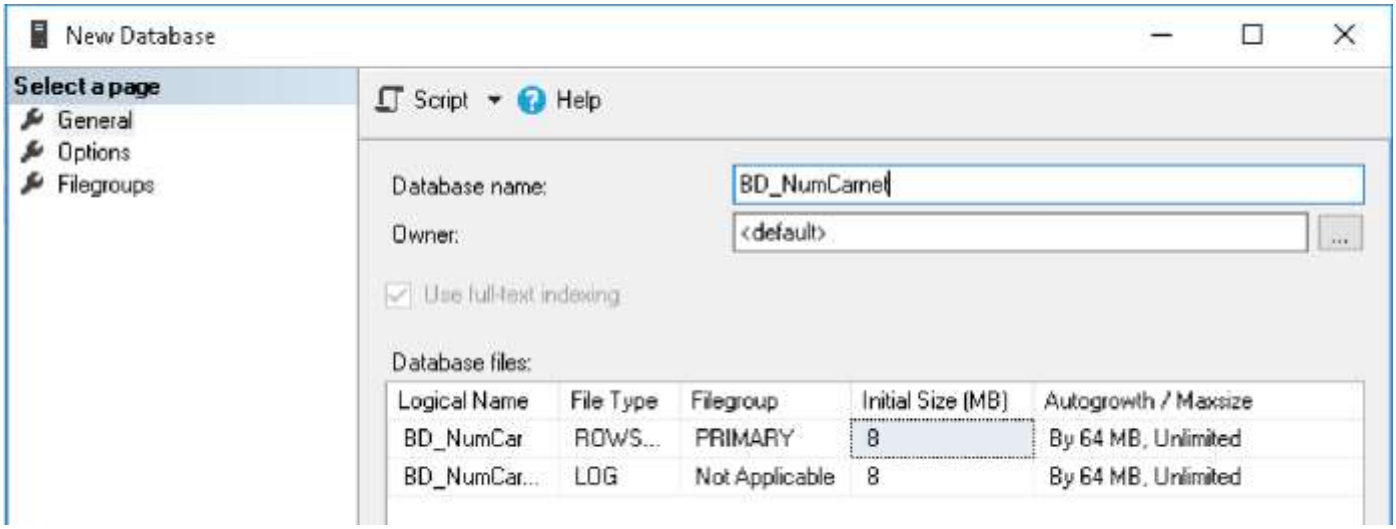
Parte 2: Manipulando una base de datos con asistente

Ejercicio 1. Crear una base de datos

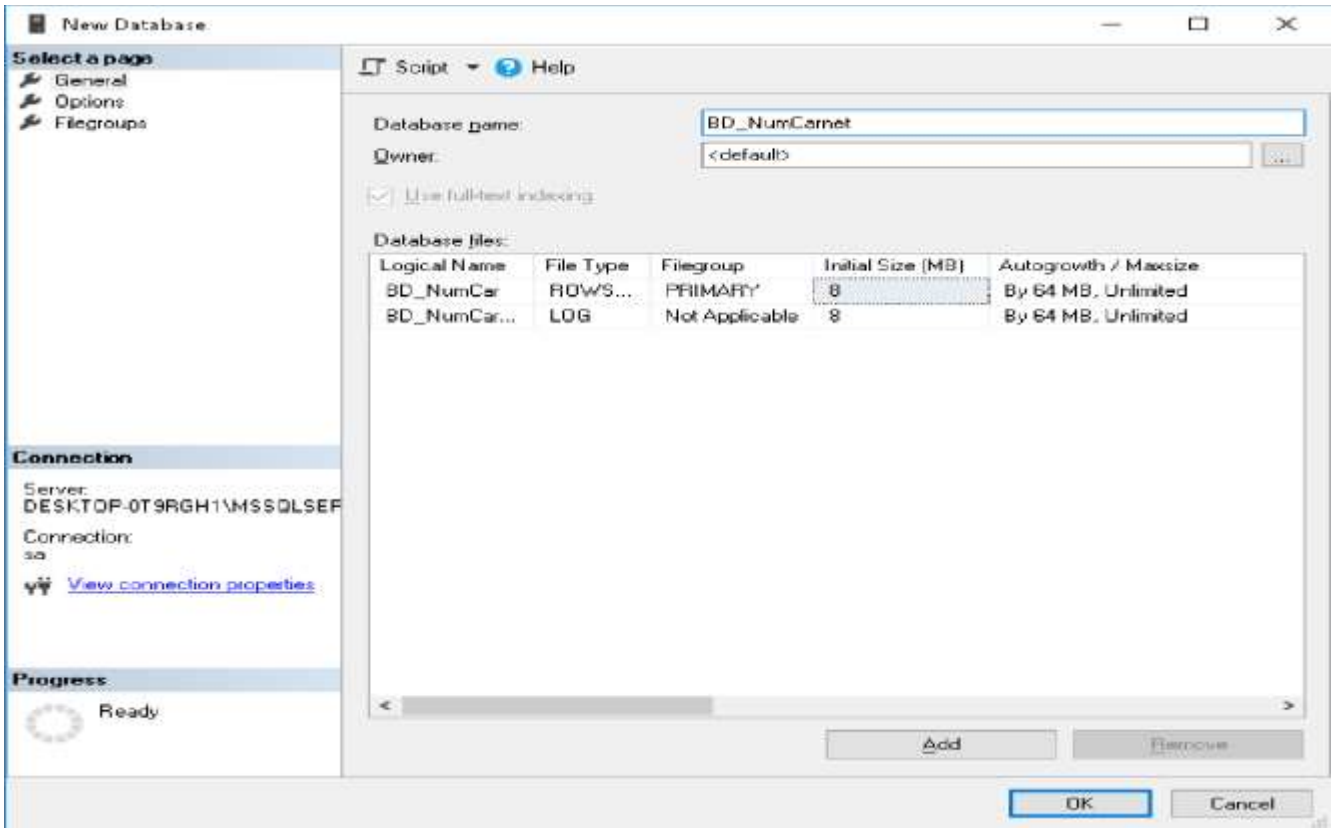
1. Hacer clic derecho en la opción **Bases de datos (Databases)** y, a continuación, en **Nueva base de datos (New Database…)**.



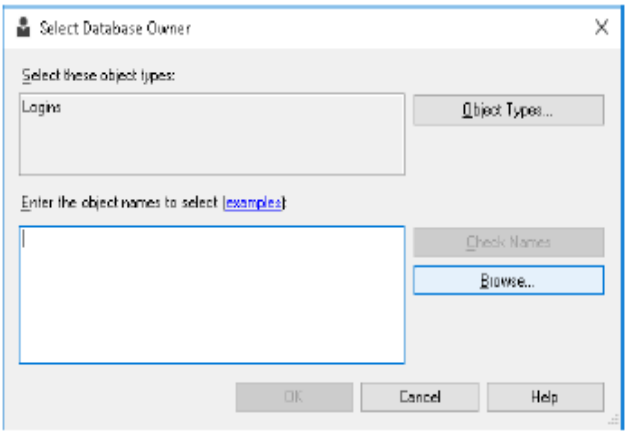
2. En la ventana **Nueva base de datos (New Database)**, especifique un nombre de base de datos.

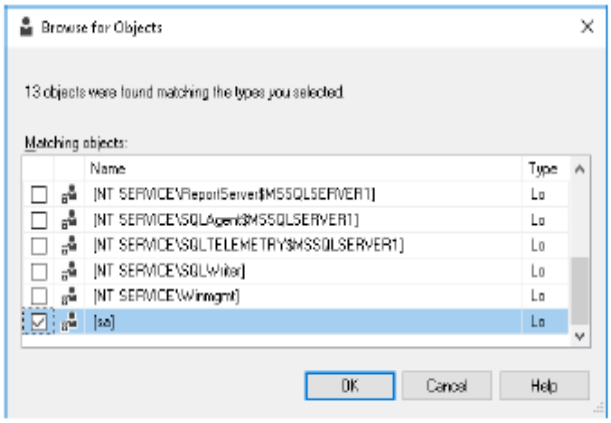


Para este ejemplo digite BD_NumCarnet, **Nota:** Sustituya NumCarnet por su número de carnet



3. Para cambiar el nombre de nombre del propietario, haga clic en (**…**) para seleccionar otro.





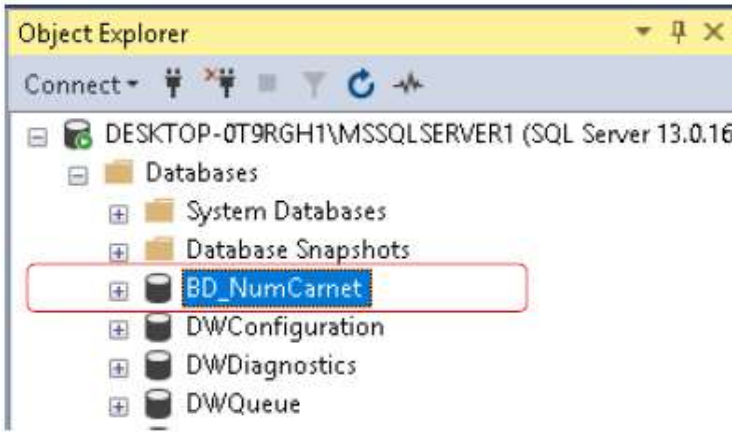
4. Haga clic en la opción Browse, buscar el nombre sa, seleccionarlo y hacer clic en OK

5. Para cambiar los valores predeterminados de los archivos de datos y de registro de transacciones principales, en la cuadrícula **Archivos de la base de datos (Database files)**, haga clic en la celda correspondiente y especifique el nuevo valor.

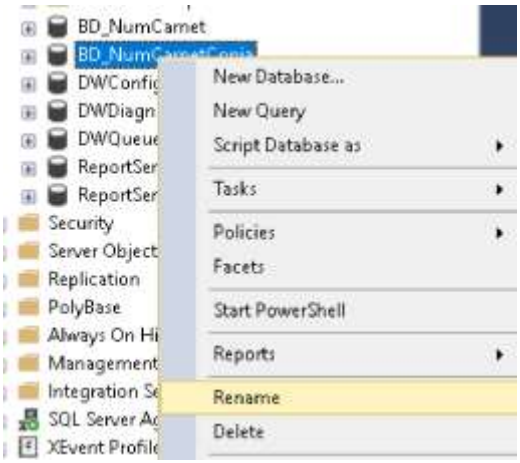
6. En la opción Initial Size (MB) cambie el valor entre 8 a 10

7. Para crear la base de datos, haga clic en **Aceptar (OK).**

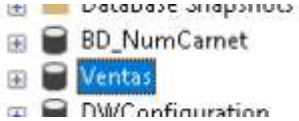
8. Expanda la carpeta donde se encuentran las bases de datos y observará su nueva base de datos:



Ejercicio 2. Cambiar nombre a una base de datos
Debe asegurarse que nadie esté utilizando esta base de datos
1. Haga clic derecho en la base de datos cuyo nombre desea cambiar (BD_NumCarnet) y, a continuación, haga clic en **Cambiar nombre (Rename).**

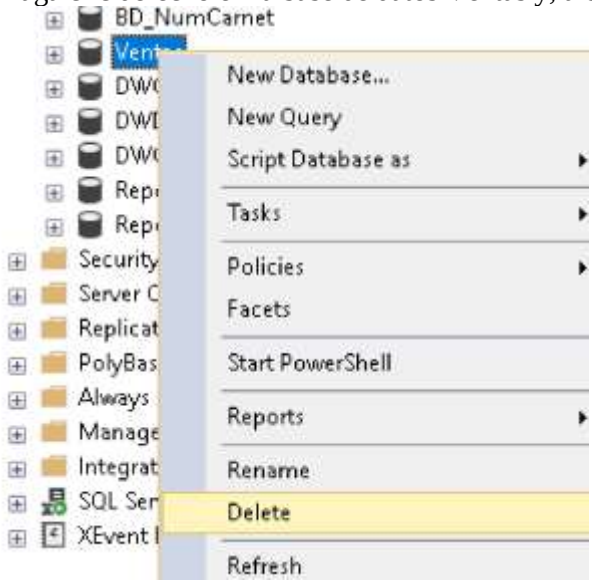


2. Escriba el nuevo nombre de la base de datos, por ejemplo, **Ventas** y presione la tecla Enter para aceptar los cambios.

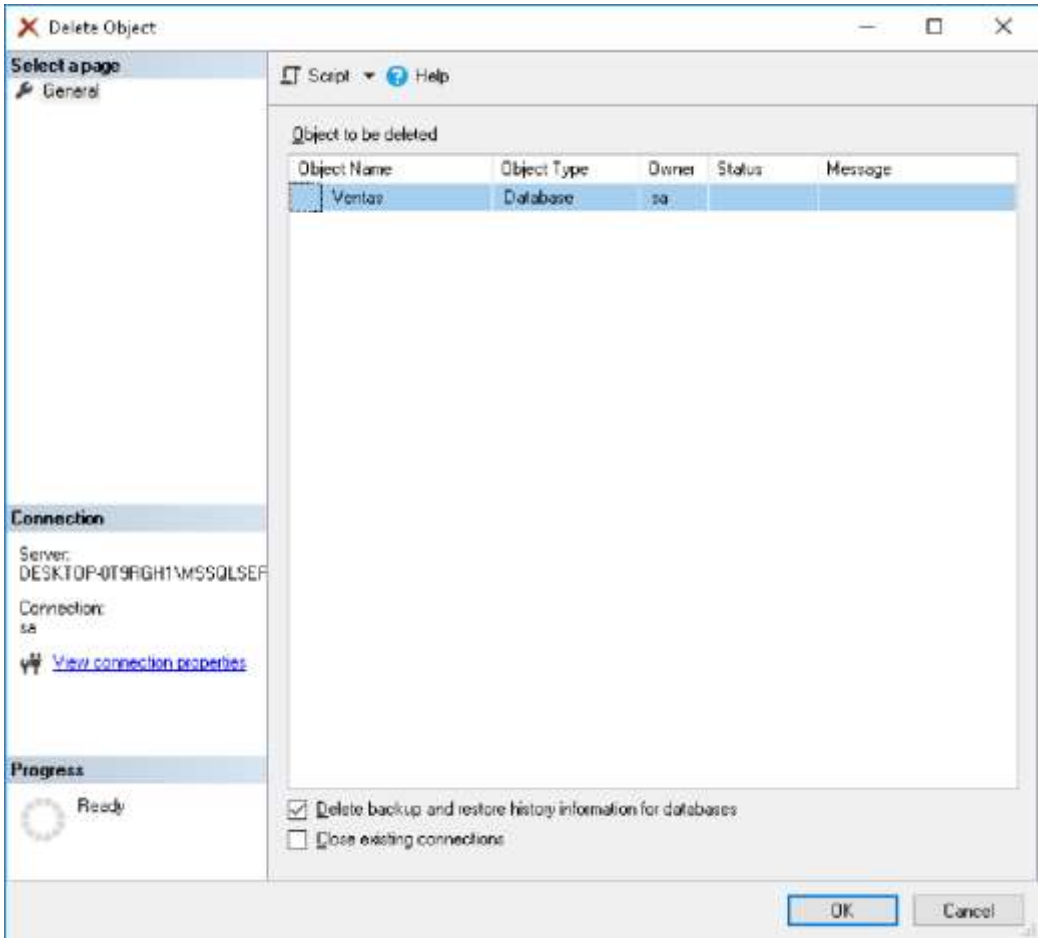


Ejercicio 3. Eliminar una base de datos

1. Haga clic derecho en la base de datos Ventas y, a continuación, haga clic en **Eliminar (Delete)**.



2. Confirme que haya seleccionado la base de datos correcta y haga clic en **Aceptar (OK)**.



Y verifique que la base de datos Ventas ya no existe en el servidor

Ejercicio 4. Adjuntar una base de datos

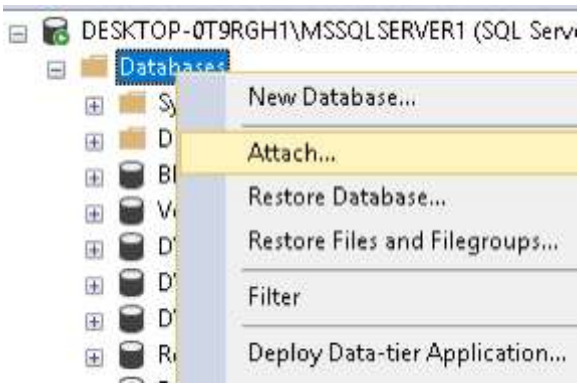
1. Copiar la base de datos ControldeVentas, la cual se encuentra en los recursos de la Guía en la página o el aula virtual de la universidad

2. En la carpeta ya descomprimida verá que se encuentran los archivos Ventas.mdf y Ventas_log.ndf

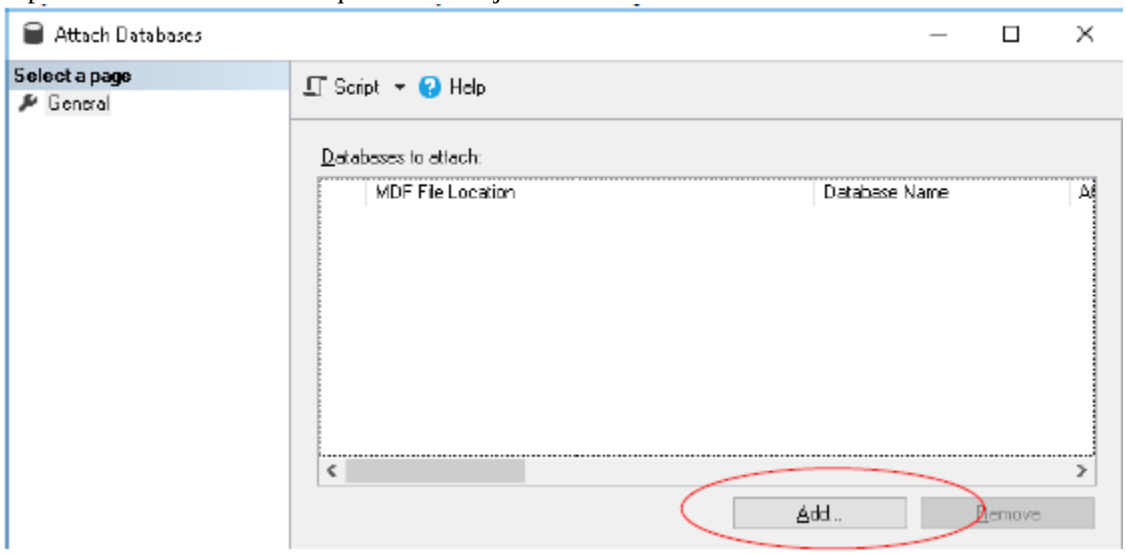
3. Copiar esos archivos en la carpeta DATA del SQL Server, puede verificar la siguiente ruta:



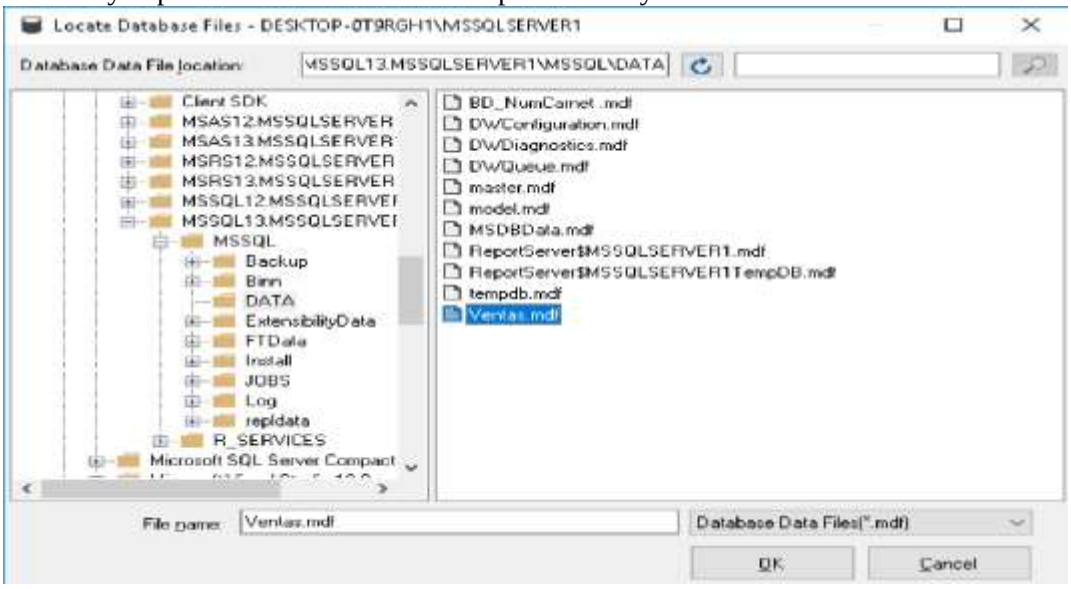
4. Haga clic derecho en Bases de datos y haga clic en Adjuntar (Attach…)



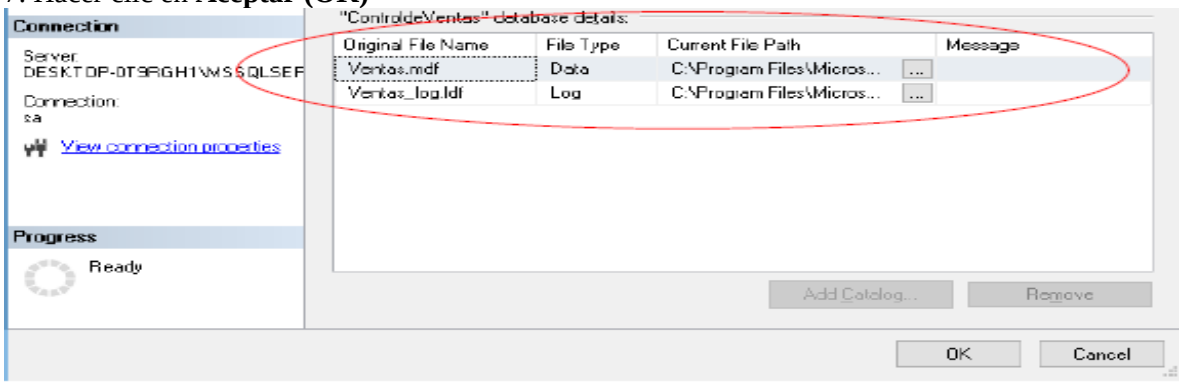
5. En el cuadro de diálogo **Adjuntar bases de datos (Attach Databasese)**, haga clic en **Agregar (Add)** para especificar la base de datos que se va a adjuntar



6. En el cuadro de diálogo Buscar archivos de base de datos, seleccione la unidad de disco en la que se halla la base de datos y expanda el árbol de directorios para buscar y seleccionar el archivo .mdf de la base de datos

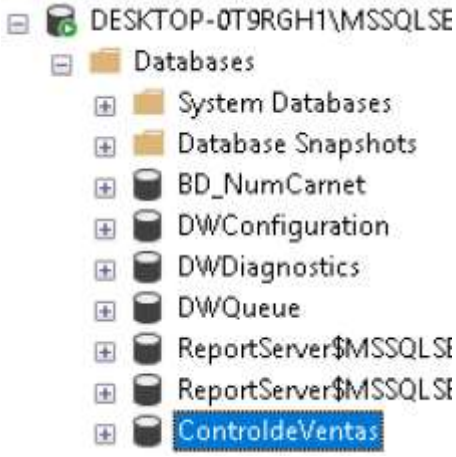


7. Hacer clic en **Aceptar (OK)**

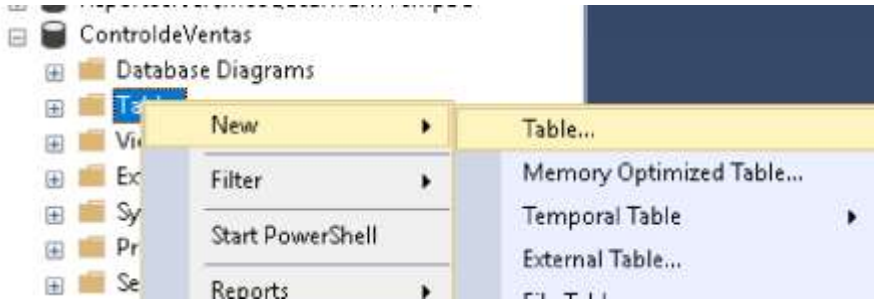


8. Hacer clic en **Aceptar (OK)** otra vez

9. Y observara la nueva base de datos agregada al servidor



Parte 3: Uso del objeto tabla en una base de datos
Ejercicio 1. Crear una tabla
Para crear una tabla nueva con el Diseñador de tablas
1. En el Explorador de objetos, haga clic derecho en el nodo **Tablas (Tables)** de la base de datos ControldeVentas y, a continuación, haga clic en **Nueva -> tabla (New -> Table)**

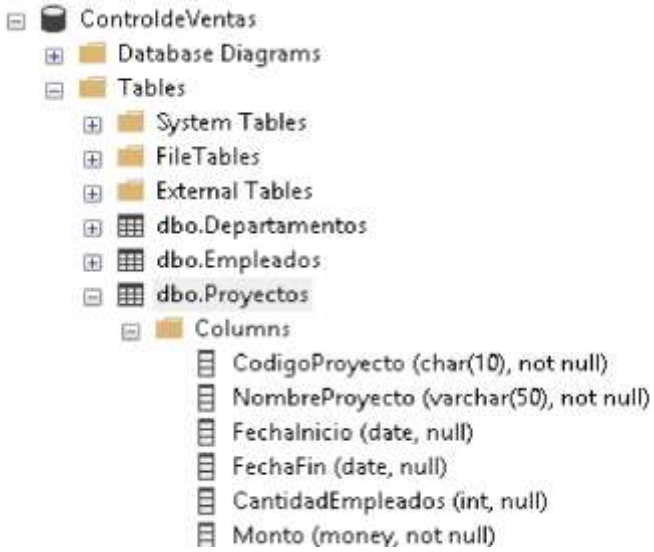


2. Escriba los nombres de columna, elija los tipos de datos y decida en cada columna si va a permitir valores NULL. (Allow Nulls) el cheque indica que va a permitir valores nulos o vacíos en la columna o campo
Crear la siguiente tabla:

Column Name	Data Type	Allow Nulls
CodigoProyecto	char(10)	☐
NombreProyecto	varchar(50)	☐
FechaInicio	date	☑
FechaFin	date	☑
CantidadEmpleados	int	☑
Monto	money	☐

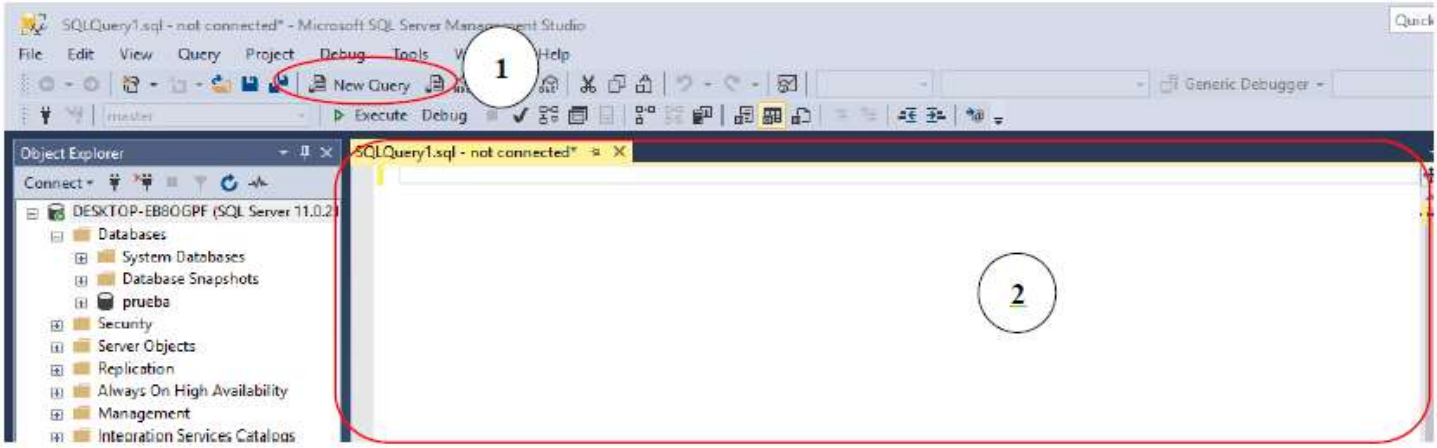
3. En el menú **Archivo (File)**, seleccione **Guardar Tabla_1 (Save Table_1)** .

4. En el cuadro de diálogo **Elegir Nombre (Choose Name)**, escriba un nombre nuevo para la tabla el cual es **Proyectos** y haga clic en **Aceptar**.



Parte 4: Manipulando una base de datos sin asistente
Ejercicio 1. Crear la base de datos

1. Agregar la opción para editar una nueva consulta, hacer clic en el botón **New Query** (1), para que se habilite un espacio en blanco (2) en donde se pueden ingresar o digitar las sentencias o consultas SQL



2. Crear en la **unidad C** una carpeta con el nombre **Guia2_SuCarnet,** para que pueda verificar la creación de los archivos .mdf y .ldf

3. Crear la base de datos **BasedeDatos_SuCarnet**, en el área de edición de **consultas SQL**, digitar la siguiente consulta:

```
--Creando la Base de Datos
USE master;
GO
CREATE DATABASE BasedeDatos_SuCarnet
ON
( NAME = BasedeDatos_SuCarnet_dat,
  FILENAME = "C:\Guia2_SuCarnet\BasedeDatos_SuCarnetdat.mdf",
  SIZE = 5,
  MAXSIZE = 20,
  FILEGROWTH = 5 )
LOG ON
( NAME = BasedeDatos_SuCarnet_log,
  FILENAME = "C:\Guia2_SuCarnet\BasedeDatos_SuCarnetdat.ldf",
  SIZE = 2,
  MAXSIZE = 10,
  FILEGROWTH = 2
)
GO
```

Nota importante: Recuerde que debe sustituir la palabra **SuCarnet** con su número de carnet

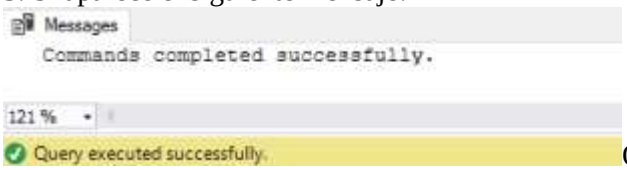
GO

- Indica a las herramientas de Microsoft SQL Server el final de un lote de instrucciones Transact-SQL.
- No es una instrucción de Transact-SQL, sino un comando que reconocen las herramientas osql e isql y el Analizador de consultas de SQL.

4. Ahora se tiene que ejecutar la consulta, tiene que hacer clic en la opción **Execute** (Ejecutar)

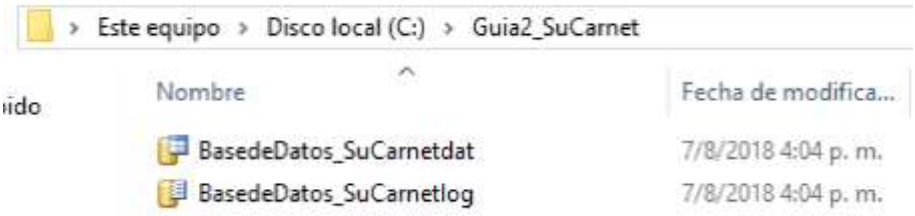


5. Si aparece el siguiente mensaje:

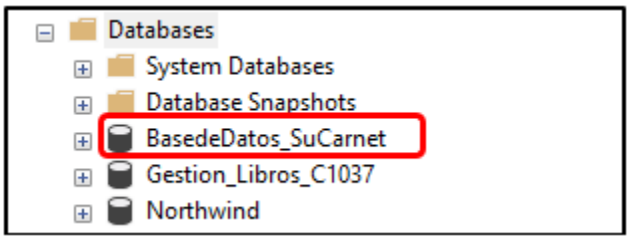


Quiere decir que la consulta se ejecutó correctamente.

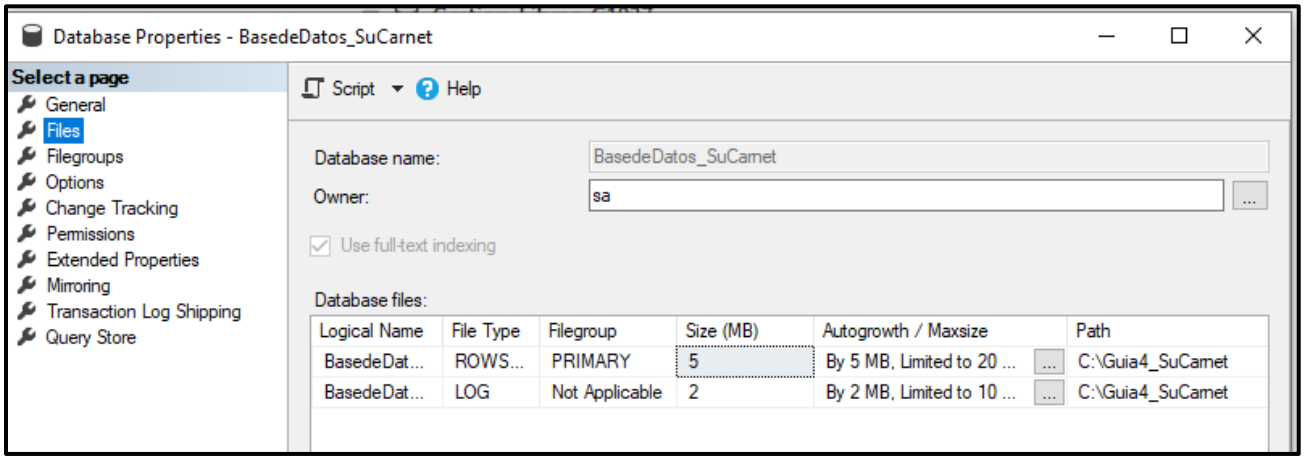
6. Revise la carpeta en la Unidad C y deberá tener los archivos .mdf y .ldf



7. Para verificar si se creó la base de datos, actualice (refresh) la carpeta **Databases** (Base de datos) y observará que se creó la nueva base de datos



8. Verifique las propiedades (properties) de la base de datos si son las que se colocaron cuando se creó la base de datos:



Ejercicio 2. Crear las tablas de la base de datos

Las tablas a crear en este ejercicio son:

Tablas	Campos
Autor	CodigoAutor Nombre (PrimerNombre y PrimerApellido) FechaNacimiento Nacionalidad
Libro	CodigoLibro Titulo ISBN AñoEdicion

	CodigoEditorial
Editorial	CodigoEditorial Nombre País
Detalle_AutorLibro	CodigoAutor CodigoLibro Fecha

1. Después de la consulta que utilizó para crear la base de datos, digite la siguiente consulta SQL:

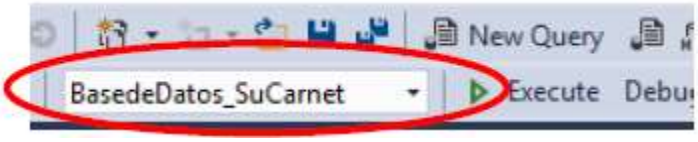
```
--Creando las Tablas
--Tabla Autor

Use BasedeDatos_SuCarnet
GO
CREATE TABLE Autor
( CodigoAutor char(5) NOT NULL,
  PrimerNombre varchar(25),
  PrimerApellido varchar(30),
  FechaNacimiento date,
  Nacionalidad varchar(35)
)
```

2. Seleccione desde el comando USE hasta el cierre del paréntesis y ejecute la consulta, si aparece el siguiente mensaje, se creó correctamente la tabla

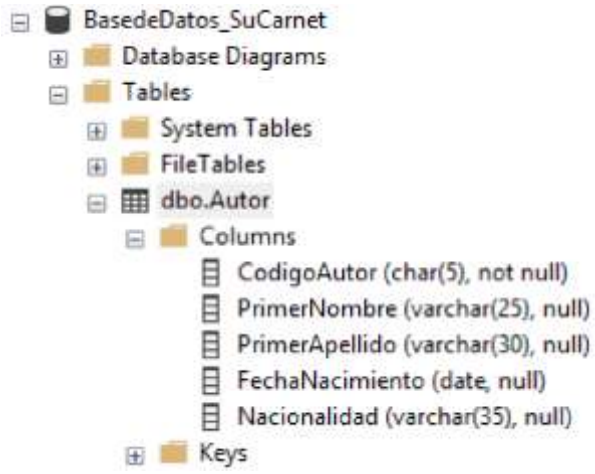


Un punto muy importante es que con la instrucción **USE**, hemos seleccionado la base de datos en la cual deseamos trabajar



Nota: La instrucción **USE** se usará las veces que nos queremos cambiar de base de datos, sino solo se utiliza una vez

3. Actualice (Refresh) su Base de datos y abra la carpeta Tables y deberá observar la tabla **Autor** creada



4. Ahora se tiene que crear la tabla **Libro**, digite después de la última instrucción SQL la siguiente consulta:

```
--Tabla Libro
CREATE TABLE Libro
(CodigoLibro char(10) NOT NULL,
 Título varchar(max),
 ISBN varchar(20) NOT NULL,
 AñoEdicion char(4),
 CodigoEditorial char(5),
)
 GO
```

Nota: Si su servidor está en inglés, todo nombre de columna que contenga caracteres que no pertenezcan al alfabeto inglés deberán ser escritos entre corchetes, por ejemplo: [AñoEdicion].

5. Seleccione la consulta que está creando la tabla Libro y ejecútela, actualice la base de datos y deberá observar la tabla agregada.

6. Crear la tabla Editorial, después de la consulta anterior, digitar la siguiente:

```
--Tabla Editorial
CREATE TABLE Editorial
(CodigoEditorial char(5) NOT NULL,
Nombre varchar(45),
Pais varchar(50),
) GO
```

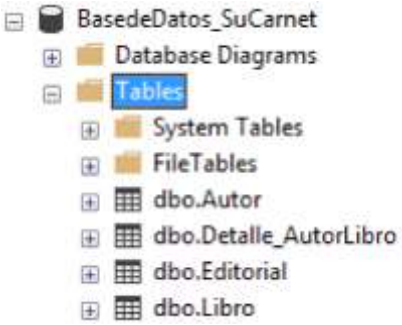
7. Ejecutar la consulta y actualice la base de datos

8. Crear la tabla **Detalle_AutorLibro**, después de la consulta anterior, digitar la siguiente:

```
--Tabla Detalle_AutorLibro
CREATE TABLE Detalle_AutorLibro
(CodigoAutor char(5),
 CodigoLibro char(10),
 Fecha date
)
```

```
GO
```

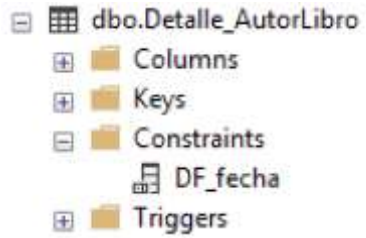
9. Ejecutar la consulta y actualice la base de datos, al final deberá tener en la base de datos BasedeDatos_SuCarnet, las cuatro tablas creadas:



Ejercicio 3. Creando restricciones: Default, Check y Unique
Restricción Default
1. Se creará una restricción Default en el campo Fecha para la tabla Detalle_AutorLibro, en la cual si el usuario no digita nada, para esta fecha que se introduzca la fecha del sistema utilizando la función **getdate()**

```
--Restriccion Default
ALTER TABLE Detalle_AutorLibro
ADD CONSTRAINT DF_fecha
DEFAULT getdate() FOR Fecha;
```

2. Actualice su tabla Detalle_AutorLibro y verifique en la carpeta Constraints la creación de la restricción Default



Restricción Check
3. Digite la siguiente consulta:

```
--Restriccion Check
ALTER TABLE Libro
ADD CONSTRAINT CK_añoedicion
CHECK (AñoEdicion > 2010);
```

4. La cual agrega una restricción Check , para el campo AñoEdicion de la tabla Libro, los datos que se introduzcan para este campo deben ser mayores al año 2010

5. Ejecute la consulta y verifique los cambios que le hizo a la columna



Restricción Unique

6. La consulta que se digitará a continuación, crea una restricción Unique para el campo ISBN de la tabla Libro, el cual se puede tomar ese campo como dato único, pero no es una clave principal ya que ese campo no se utiliza para crear relaciones entre tablas

7. Digite la siguiente consulta:

```
--Restriccion Unique
ALTER TABLE Libro
ADD CONSTRAINT U_isbn
UNIQUE (ISBN)
```

8. Actualice la tabla Libro y vea que agrego la restricción Unique



9. Hasta aquí ha creado las restricciones asegurando la calidad de los datos

Ejercicio 4. Usando los comandos DROP DATABASE y DROP TABLE

El comando DROP se utiliza para quitar objetos existentes como base de datos, tablas, usuarios, vistas, triggers, procedimiento almacenado etc.

1. Digitar la siguiente consulta:

```
--Uso del Comando DROP
CREATE DATABASE GUIA2_Sucarnet
GO

USE GUIA2_Sucarnet

GO

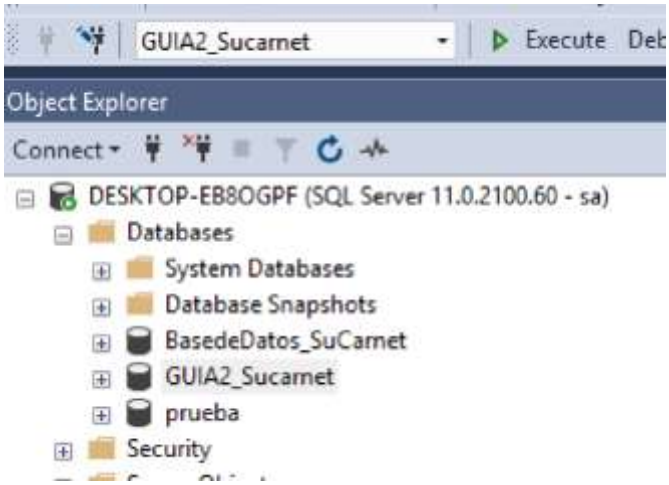
CREATE TABLE tabla
(campo1 int,
campo2 varchar(5)
)
```

Nota importante: Recuerde cambiar la palabra **Sucarnet** por su número de carnet

2. Seleccione todas las sentencias SQL que digito en el punto 1 y ejecútelas

GO es un signo de finalización de un lote de sentencias. No es una sentencia, es un comando. El lote de sentencias está compuesto por todas las sentencias antes de GO o todas las sentencias entre dos GO

3.Verifique que creo la base de datos y la tabla dentro de la base de datos **Guia2_Sucarnet**, como también que ya está haciendo uso de ella



4. Ahora vamos a eliminar la tabla de la base de datos

5. Digitar la siguiente consulta

```
--Eliminando la tabla
DROP TABLE tabla
```

6. Actualice su base de datos y ya no debe tener la tabla
7. Vamos a eliminar ahora la base de datos
8. Digite la siguiente consulta:

```
--Eliminando la base de datos
        USE Master
        GO
        DROP DATABASE Guia2_Sucarnet
```

9. Actualice la carpeta Databases y ya no debe estar la base de datos que creo en este ejercicio

Ejercicio 5. Guardar las consultas SQL

1. Ahora vamos a guardar las consultas en un archivo que tiene la extensión .sql esto para en cualquier momento haga uso de las consultas

2. Hacer clic en el menú **File** y seleccione la opción **Save SQLQuery1.sql As…** busque la carpeta que creo al inicio del ejercicio 1

3. Y guarde el archivo como Guia2.sql

4. Verifique que creo el archivo .sql, ese es el archivo que le enviará al docente.

V. Ejercicios Complementarios

Ejercicio 1.
1. Crear una base de datos con el siguiente nombre: Control_Inscripciones
2. Crear las siguientes tablas:

Tabla alumno:
- Carnet
- Nombres
- Apellidos
- Dirección
- Correo
- Teléfono
- Fecha de nacimiento

Tabla materia:
- Código
- Nombre
- UV
- Prerrequisitos
- Ciclo

Tabla inscripción:
- Código_materia
- Carnet
- Matricula

1. Identifique los tipos de datos que se pueden implementar en cada uno de los campos
2. Copiar los archivos mdf y ldf

Ejercicio 2
En una nueva área de edición de consultas crear la siguiente base de datos en SQL Server:
Nombre de la base de datos: **Hotel_SuCarnet**
Tablas:

Tablas	Campos
Hotel	CodigoHotel Nombre Direccion SitioWeb
Reservacion	CodigoReserva CodigoHotel CodigoHuesped CodigoHabitacion FechaInicio FechaFin
Habitacion	CodigoHabitacion CodigoHotel Tipo Precio
Huesped	CodigoHuesped Nombre Email Dirección Telefono

Haciendo uso de TRANSACT – SQL crear:
1. La base de datos
2. Las tablas con sus correspondientes campos y propiedades

3. Crear las siguientes restricciones en los campos:
    a. Unique:
        i. Tabla Hotel (Nombre, SitioWeb)
        ii. Tabla Huesped (E-mail)
    b. Check:
        i. Tabla Habitacion (Precio mayor que 25, Tipo: Doble, individual)
        ii. Tabla Reservacion (Fecha fin tiene que ser mayor a la fecha de inicio)
    c. Default:
        i. Tabla Reservacion (Fecha inicio por defecto puede ser la fecha actual del sistema)

4. Guarde el archivo de las consultas con el nombre **Consultas_Hotel.sql,** y enviarlo al docente
Nota: Las relaciones entre tablas se verán en la siguiente guía

VI. Fuente de consulta

1. La Biblia de SQL Server
2005 Madrid, España:
Anaya, 2006
Autor: Mike Gundelerloy y Joseph L. Jorden
Biblioteca UDB – Clasificación: 005.361 G975

2006

2. Microsoft SQL Server 2008: Guía del Administrador Madrid, España: ANAYA, 2009
Autor: William Stanek
Biblioteca UDB – Clasificación: 005.361 S784 2009